

Urgent and unusual situations can be allocated an interrupt and dealt with immediately while normal sensing proceeds asynchronous to the control logic

or executing the most current command from the control logic (in the case of outputs). The fewer the timing or order-of-execution constraints, the better. So identify all possible timing dependencies and try to eliminate them. Looking at the temperature block, one possible design is for the temperature conversion to be initiated by the control logic block whenever it wants a new temperature. This surely provides the most up-to-date value, but at what cost? Either the control logic will have to wait for a conversion to complete and slow down the whole system, or the device providing this conversion must be fast enough to not slow the system, which means a more expensive solution.

Take a deeper look and separate the normal operating conditions (for instance, the oven is heating and the new temperature will change the current heating command) from the conditions requiring immediate action (like sensing a dangerously high temperature, perhaps caused by an electrical short). Urgent and unusual situations can be allocated an interrupt and dealt with immediately while normal sensing proceeds asynchronous to the control logic. By doing a similar situational analysis, you can set the timing boundaries for each block, defining the worst-case timing and ensuring it meets product requirements while giving each functional block maximum room to operate.

Step 4: Enable swappable features

From steps 2 and 3, you can see that maximum flexibility is being designed-in for each block. This step concentrates on taking that concept to all aspects of the product. Each feature (at least the features affecting the embedded design) can be treated in a similar fashion. Should we use a set of tactile keys for the user inputs or modern ca-

pacitive sensors? Should the memory for the recipe database be 8 kB of internal Flash or something removable like an SD card that a user could upgrade with new recipes (by removing the card, plugging it into a standard card reader on their computer and accessing our website)? Does the convection system use a single fan or multiple fans with several temperature zones?

For every feature, define a wrapper that allows the details of each feature to change under the hood while the system sees a consistent interface. Just like the example for the temperature, a similar approach can be applied to 'wrapping' every feature. One way to look at this is to create feature objects that contain the algorithms and details but present the results in a consistent and easy-to-use way to the control logic. For instance, if the display might need to display English, French, Chinese or Korean prompts, the feature object would have a property for 'display language' and function 'void setLanguage(char newLang)' to change the language displayed. Add more languages by updating the low-level display functions, not the control logic.

What about a major change like going from a simple one-temperature-zone design to multiple-zones? Even if such a major change were to occur, the affected pieces in the temperature, heater and convection blocks would get updated and perhaps a new control function would be needed—but at least everything wouldn't change. You can go one better than that by having the 'what if' brainstorming session and driving any potentially affected pieces into a feature block. The goal of this step is to create a set of pieces that can undergo radical changes with limited impact to the overall system.

An additional side benefit is that feature objects are highly reusable when they are designed to shield the

rest of the system from their own changes. The same flexibility of change within the system is provided to future systems. If you have multiple teams working on different but related products, this approach provides the ability to share technology and innovation, especially if the products share a similar structure to begin with.

Object-oriented design benefits without objective languages

Careful readers will have noticed that by following the four steps, it looks like I have directed this design to become 'object-oriented'—partitioned into objects each with its specific data or implementation hidden. Guilty as charged.

The benefits I assert you will gain by this approach—such as limiting the effect of any single change on the rest of the design and reusability—have been touted for decades by proponents of object-oriented design and programming. And most new programming languages and variants structurally support object-based design.

But an object-oriented design does not succeed or fail based upon language selection—no language by itself will turn code churned out without a strong design into a success. Similarly, you can achieve the benefits described with any language, and since 'C' is in common-use in embedded design, I chose to think of this design approach using 'C'. In fact, one of the best object-oriented designs I have worked on used Intel PL/M programming language on an 80C86 processor and was successful entirely due to the design; the language was barely one step up from Assembly.

While there is no miracle cure for late-cycle requirement changes, the steps I have laid out above will lead to a successful object-based design that can integrate requirement changes as they occur with the least impact to the overall system and schedule. ●

The author is from Cypress Semiconductor Corp.